Ministerul Educației și Cercetării al Republicii Moldova
**Universitatea Tehnică a Moldovei**
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică
Program: Software Engineering

# Report on laboratory work no. 6

Course: Operating Systems

## Topic: Inside the xv6 File System

*Track B: build your own small OS, xv6*

Done by:
st. gr. FAF-241,
**Serghei Barladean**

Checked by:
**Patricia Reitman**

Chișinău – 2026

## Contents

Purpose of the Work....3
Part 1. Reading What xv6 Shows....3
1.1. Work Done....3
1.2. Observations....5
Part 2. Writing mystat....5
2.1. Work Done....5
2.2. Observations....8
Part 3. Measuring the Limits....8
3.1. Work Done....8
3.2. Observations....9
Part 4. Writing find....10
4.1. Work Done....10
4.2. Observations....12
Conclusions....13

## Purpose of the Work

The purpose of this laboratory work is to look inside the xv6 file system. It has the same ideas as ext4 (inodes, directories as tables of names, hard links, a log) in a few hundred lines of C. The work reads what xv6 shows to the user, writes an own program `mystat` on top of the `stat()` system call, measures the limits written in `kernel/fs.h`, and writes the program `find`, which walks the directory tree recursively.

The work was done on Ubuntu 24.04 (aarch64) in a Multipass virtual machine on macOS. xv6 (RISC-V) was built in `~/xv6-riscv` and run in the QEMU emulator; both new programs were added to the `UPROGS` list of the Makefile. Every test session in xv6 started from a freshly built disk image (`fs.img` was removed and rebuilt before the run), so earlier experiments could not influence the results. The figures are taken from the recorded terminal sessions.

## Part 1. Reading What xv6 Shows

### 1.1. Work Done

After `make qemu` the shell was asked for the contents of the root directory, a new directory `a` was created and listed:

```
ls
mkdir a
ls a
```

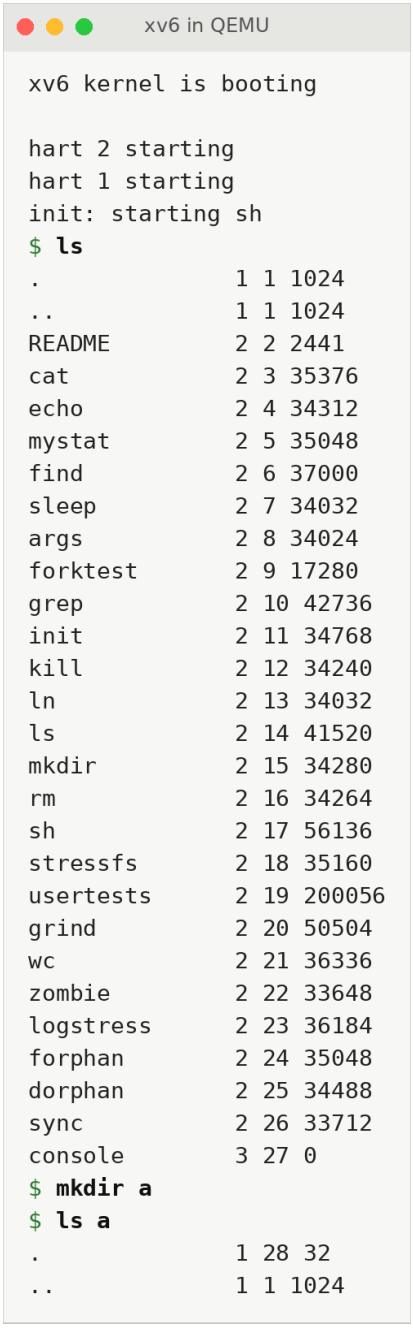



*Figure 1.1 – The root directory of xv6 and the new directory a*

Each line of `ls` has the form name, type, inode number, size in bytes. The meaning of the type was looked up in the kernel header:

```
grep -n "T_DIR\|T_FILE\|T_DEVICE" kernel/stat.h
```

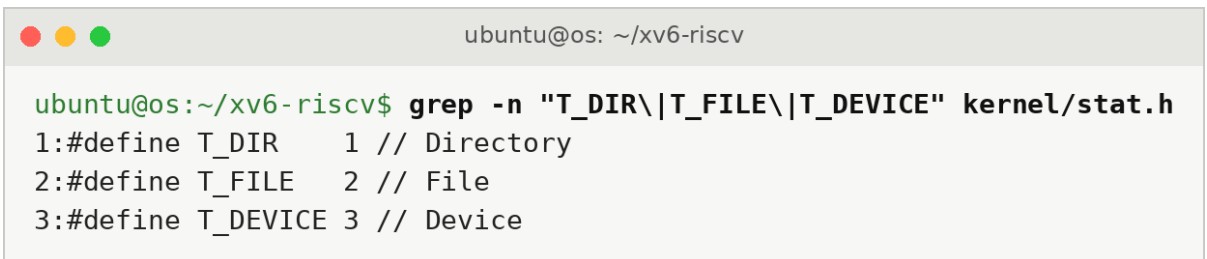



*Figure 1.2 – The file types defined in kernel/stat.h*

### 1.2. Observations

**(1)** In `kernel/stat.h` type 1 is `T_DIR` (directory), type 2 is `T_FILE` (regular file) and type 3 is `T_DEVICE` (device). In Figure 1.1 the entries `.` and `..` have type 1, `README`, `cat` and the other programs have type 2, and `console` has type 3.

**(2)** The root directory has inode number 1. Both `.` and `..` show inode 1 there, because the root has no parent and its `..` points to itself.

**(3)** One directory entry has a fixed size of 16 bytes: a 2-byte inode number and a name of `DIRSIZ` = 14 bytes. The new directory `a` has size 32, that is two entries: `.` (inode 28, itself) and `..` (inode 1, the root, its parent).

## Part 2. Writing mystat

### 2.1. Work Done

The program `user/mystat.c` calls `stat()` for every argument and prints the result in one line. If `stat()` fails, the message goes to descriptor 2 and the loop continues with the next argument:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  if(t == T_DIR)
    return "directory";
  if(t == T_FILE)
    return "file";
  if(t == T_DEVICE)
    return "device";
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;

  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

The function `typename` turns the number from `st.type` into a word. The line `$U/_mystat\` was added to the `UPROGS` list of the Makefile together with the line for the second program of this laboratory work, `find` (Figure 2.1).




*Figure 2.1 – The new programs in the UPROGS list of the Makefile*

The program was used to watch a hard link at work: `README` got a second name `r2`, then the name `README` was removed:

```
mystat README . console
ln README r2
mystat README r2
rm README
mystat r2
cat r2 | wc
```

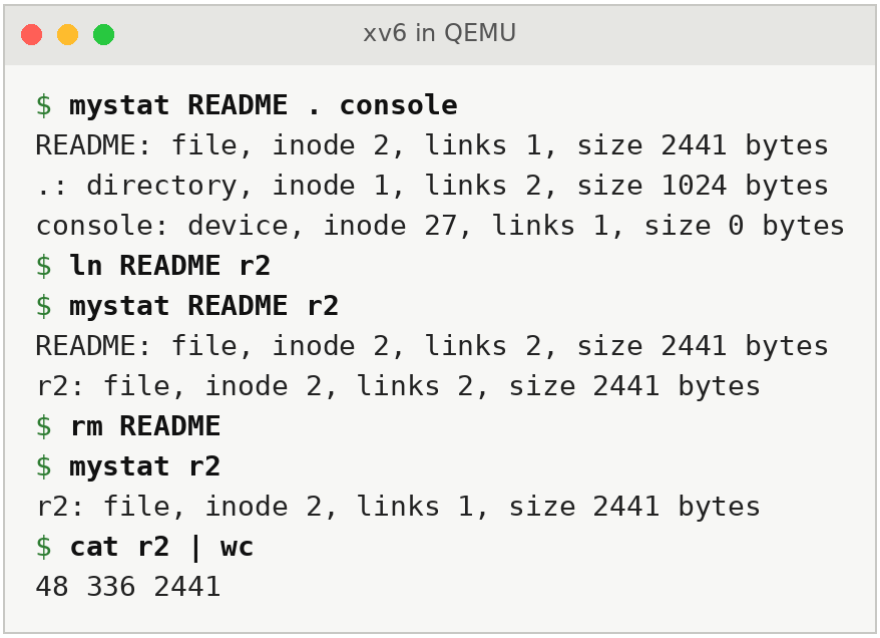



*Figure 2.2 – mystat before and after ln and rm*

Then the link count was searched in the kernel sources, first in `kernel/sysfile.c` (the system calls) and then in `kernel/fs.c`:

```
grep -n "nlink" kernel/sysfile.c
grep -n "nlink" kernel/fs.c
```

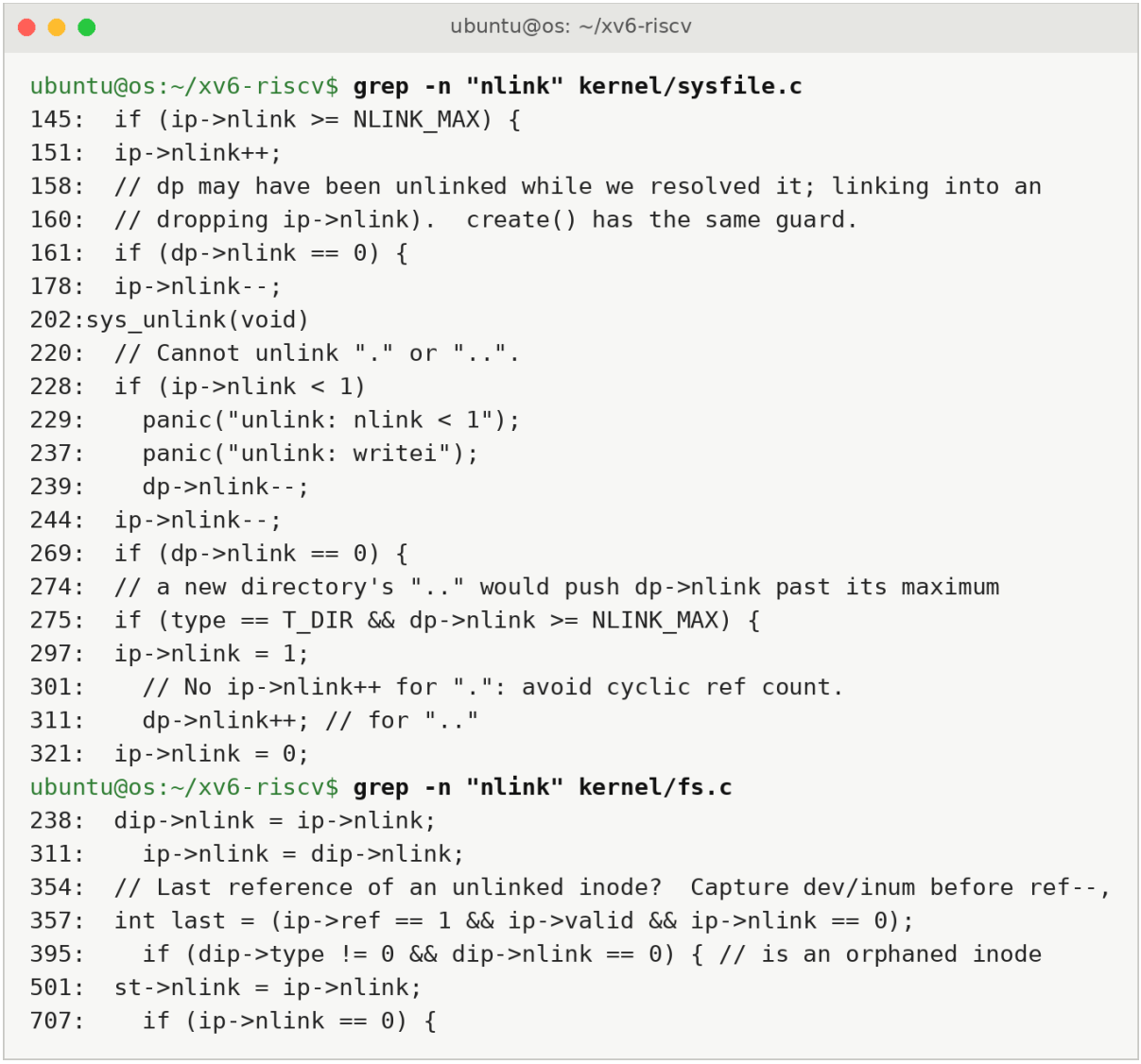



*Figure 2.3 – Every use of nlink in sysfile.c and fs.c*

Finally the on-disk inode was printed:

```
sed -n '/^struct dinode/,/^};/p' kernel/fs.h
```

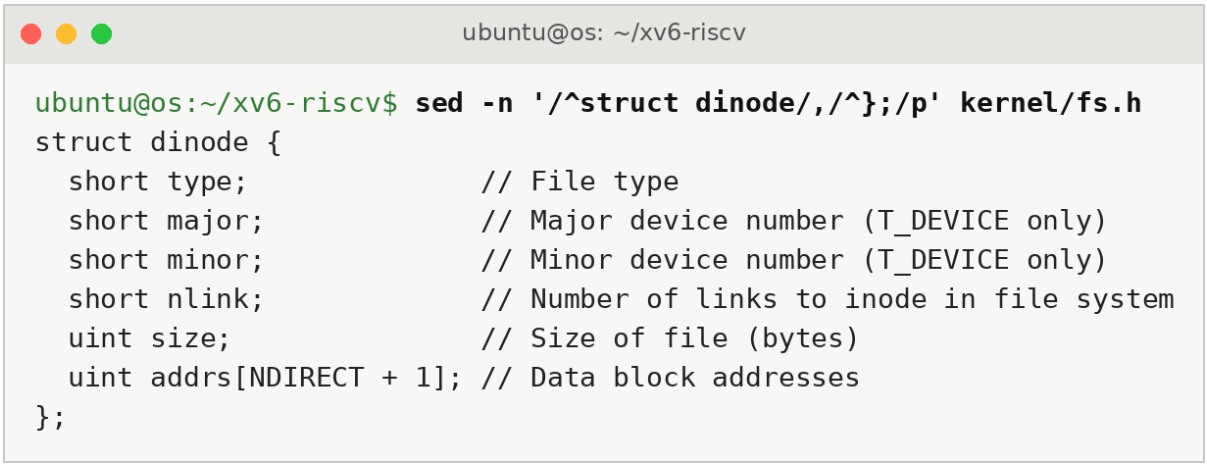



*Figure 2.4 – The structure struct dinode from kernel/fs.h*

### 2.2. Observations

**(1)** After `ln README r2` both names showed the same inode 2, the same size 2441 bytes and link count 2. After `rm README` the name `r2` still showed inode 2 and size 2441, but the link count was 1, and `cat r2 | wc` printed `48 336 2441`, which are the lines, words and bytes of the old `README`. So only the number of names changed (2 to 1, and the name `README` disappeared); the inode, the size and the data did not change. In this run the root directory showed 2 links.

**(2)** The link count is lowered in `sys_unlink` of `kernel/sysfile.c`, in line 244: `ip->nlink--;` (line 239, `dp->nlink--;`, does the same for the parent when a directory is removed, because its `..` disappears). The data blocks are not freed by `unlink` itself. They are freed when the last reference to the inode is released and the link count is 0; this test is in `iput` in `kernel/fs.c` near line 357 (`ip->ref == 1 && ip->valid && ip->nlink == 0`, the comment there says `Last reference of an unlinked inode?`). A file that is still open therefore survives `rm`, as in Lab 5.

**(3)** An xv6 inode has these fields: `type`, `major`, `minor` (only for devices), `nlink`, `size` and `addrs[NDIRECT+1]`, the addresses of the data blocks. There is no owner, no mode and no time, so xv6 has no permissions. The type can only be directory, file or device, so there are no symbolic links either.

## Part 3. Measuring the Limits

### 3.1. Work Done

The constants that describe the file system were taken from `kernel/fs.h`:

```
grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ"
kernel/fs.h
```

```
ubuntu@os: ~/xv6-riscv
ubuntu@os:~/xv6-riscv$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" k
ernel/fs.h
5:#define BSIZE  1024 // block size
26:#define NDIRECT  12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
```

*Figure 3.1 – BSIZE, NDIRECT, NINDIRECT and DIRSIZ*

The layout of the disk is printed by `mkfs` while the image is built; the line with `nmeta` was selected. The usual messages of `make clean` are visible above it:

```
make clean ; make fs.img 2>&1 | grep nmeta
```

```
ubuntu@os: ~/xv6-riscv
ubuntu@os:~/xv6-riscv$ make clean ; make fs.img 2>&1 | grep nmeta
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*/*.o */*.d */*.asm */*.sym \
kernel/kernel fs.img \
mkfs/mkfs .gdbinit \
        user/usys.S \
user/_cat user/_echo user/_mystat user/_find user/_sleep user/_args user/_forktest user/_grep us
er/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertes
ts user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_sync
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
```

*Figure 3.2 – The disk layout line printed by mkfs*

In xv6 a directory with a very long name was created and listed:

```
mkdir averyveryverylongname
ls
```

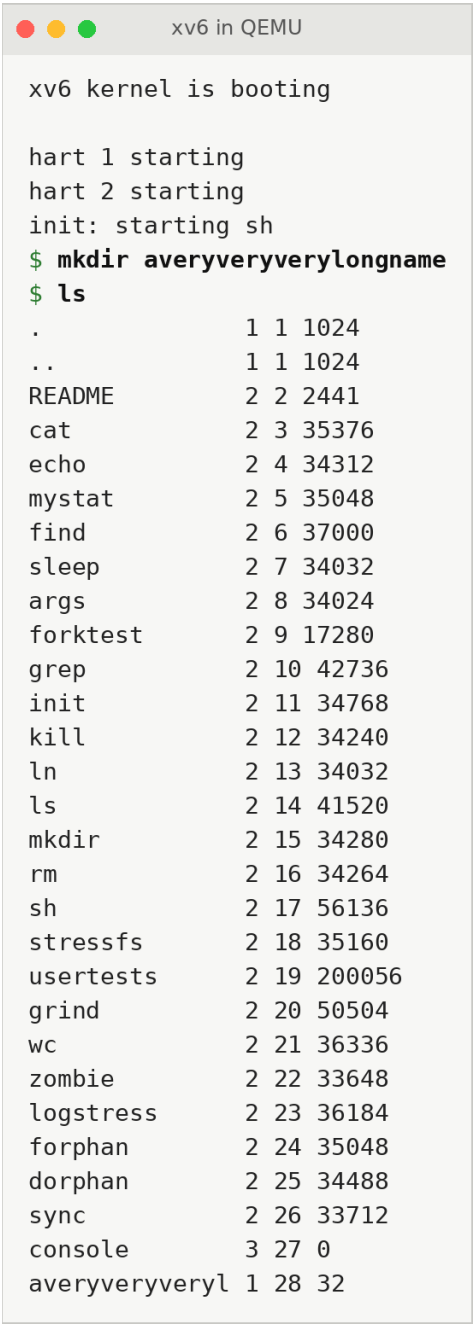



*Figure 3.3 – A directory with a name longer than 14 characters*

### 3.2. Observations

**(1)** `ls` shows `averyveryveryl`, only the first 14 characters of the name (the entry has inode 28). The constant that explains it is `DIRSIZ`, which is 14: a name in a directory entry has no more place. The command `mkdir` did not fail; the name was cut when the entry was stored.

**(2)** `BSIZE` is 1024 and `NDIRECT` is 12. The indirect block holds `NINDIRECT = BSIZE / sizeof(uint) = 1024 / 4 = 256` block numbers. The largest file is therefore (12 + 256) × 1024 = 268 × 1024 = 274 432 bytes, about 268 KB.

**(3)** The line says `nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000`. The whole disk has 2000 blocks. The log takes 31 blocks, the inodes take 13 blocks, 1 block is the bitmap, and 1 + 1 blocks are the boot and the super block; together these are the 47 metadata blocks (1 + 1 + 31 + 13 + 1 = 47). The remaining 2000 − 47 = 1953 blocks are for data. An inode on disk has 64 bytes (4 shorts, one `uint` and 13 addresses), so one block holds 16 inodes and 13 blocks hold 208.

## Part 4. Writing find

### 4.1. Work Done

The program `user/find.c` was written starting from the way `ls.c` reads a directory. The function `find` opens the directory, reads the entries one by one, builds the full path in a buffer, calls `stat` on it, prints the path if the last component is the wanted name, and calls itself for every subdirectory:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/fs.h"
#include "user/user.h"

int showino;

void
find(char *path, char *name)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, 0)) < 0){
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }
  if(fstat(fd, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }
  if(st.type != T_DIR){
    close(fd);
    return;
  }
  if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)){
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }
  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';
  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)
      continue;
    if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
```

```
      continue;
    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;
    if(stat(buf, &st) < 0){
      fprintf(2, "find: cannot stat %s\n", buf);
      continue;
    }
    if(strcmp(p, name) == 0){
      if(showino)
        printf("%s (inode %d)\n", buf, st.ino);
      else
        printf("%s\n", buf);
    }
    if(st.type == T_DIR)
      find(buf, name);
  }
  close(fd);
}

int
main(int argc, char *argv[])
{
  if(argc < 3){
    fprintf(2, "usage: find <dir> <name> [-n]\n");
    exit(1);
  }
  if(argc == 4 && strcmp(argv[3], "-n") == 0)
    showino = 1;
  find(argv[1], argv[2]);
  exit(0);
}
```

Some details of the program: entries with `inum == 0` are free slots and are skipped; `.` and `..` are never entered, otherwise the recursion would never end; the name from the entry is copied with `memmove` and a zero byte is put after `DIRSIZ` characters, because a name of 14 characters has no terminator on the disk; the comparison with `strcmp` is done before the recursion, so a directory with the wanted name is printed, too; every directory is closed after it was read, since a process may keep only 16 files open. With the third argument `-n` the program prints the inode number next to each path.

The test was run on a fresh disk, as in the handout:

```
mkdir a
mkdir a/b
echo x > a/b/r2
echo y > a/r2
ln cat r2
find . r2
find a b
```

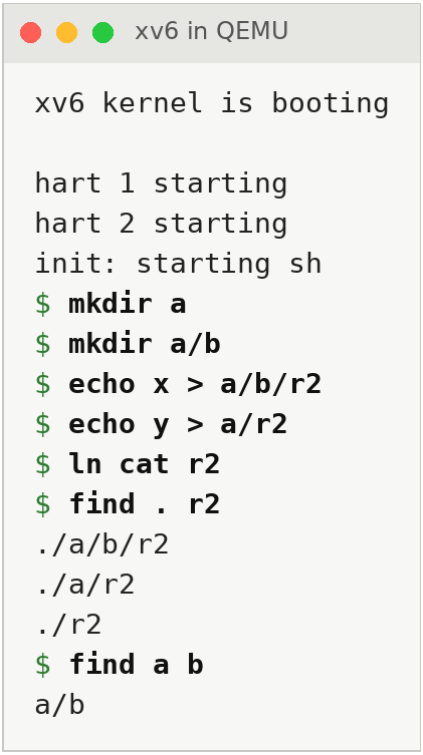



*Figure 4.1 – Testing find on a fresh disk*

The optional task was done with the third argument; `mystat` was used to compare the inode of `cat` and `r2`:

```
find . r2 -n
mystat cat r2
```

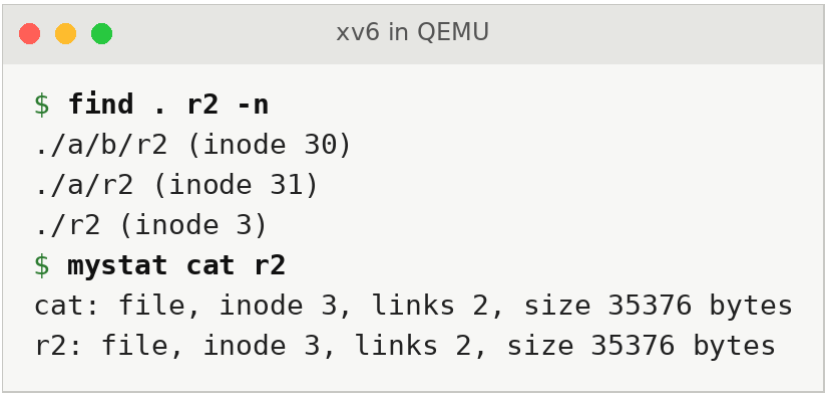



*Figure 4.2 – find with -n and the inode of cat*

### 4.2. Observations

**(1)** `find . r2` printed three paths: `./a/b/r2`, `./a/r2` and `./r2`, the same lines as in the handout. `find a b` printed `a/b`, a directory, so a match is reported for directories as well as for files.

**(2)** With `-n` the three files have the inodes 30, 31 and 3. Only `./r2` is the same file as `cat`: `cat` has inode 3 and link count 2, and `r2` has inode 3 and link count 2 (Figure 4.2), because `ln cat r2` made a second name for the program file. The other two `r2` files were created with `echo` and have their own inodes, 30 and 31.

## Conclusions

In this laboratory work the xv6 file system was examined from the user side and from the kernel side. The output of `ls` and of `mystat` showed the same inode structure that Linux has: a name is an entry in a directory, the file is an inode with a link count, and a hard link adds a second name for the same inode without copying the data. The kernel code showed where the count is lowered and when the blocks are freed.

The numbers in `kernel/fs.h` explain the visible limits. A name has at most 14 characters, a file at most 274 432 bytes (12 direct blocks and one indirect block of 256 numbers), and the disk image of 2000 blocks spends 47 of them on metadata. Neither permissions nor symbolic links exist, because the inode has no fields for them.

The programs `mystat` and `find` were written, added to the `UPROGS` list and tested in xv6. Writing `find` showed how a recursive walk over the tree is done with only `open`, `read`, `stat` and `close`, and that details like the skipped `.` and `..`, the free slots and the missing terminator of a 14-character name decide whether the program works.